

Why is passing arguments so complicated? The short answer is that there are different situations where you'll want the different behaviors. Operating on copies of the data is quick when the data is small. It allows you to have different contexts in the different functions that won't interfere with one another. It makes it easier to understand what will happen in a function because nothing can change the values of its variables except the instructions in the function itself. In other cases, it's faster to just overwrite the existing data, especially if it is very large and copying it would take a lot of time. Sharing the data structure does, however, lead to many bugs (errors) when programmers forget that some function can modify the data via a reference. As a general rule, you want to avoid designs that change (or **mutate**) the values in data that is shared by references. When you do design code to change values, make sure that the changes that can happen are well documented.

Back to the linked list, we need to use references to link together the elements. This approach solves the self-referential data structure paradox and enables the use of data structures that grow and shrink as the program runs. You can add `Link` objects as you need them, instead of trying to estimate how many are needed at the beginning of the program and allocating space for all of them in an array or some other structure.

When using references, you can think of them as their own data type. They are not integers even though most languages implement them internally as a kind of number. You can't do operations on them such as add two references together or check to see which one is larger. In general, there are no predefined values for them (like `False` and `True` for the Boolean type). There are a few special cases like the constant `null` in Java which indicates the absence of a reference. The one operation you can do is check whether two references point to the exact same object. In Python, you can compare references with the `is` operator. Let's look at some examples:

```
$ python3
>>> x = 10
>>> lx = [10]
>>> ly = [5 + 5]
>>> print(x, lx, ly)
10 [10] [10]
>>> x is lx
False
>>> lx is lx
True
>>> lx is ly
```

```
False
>>> lx == ly
True
>>> lx[0] is ly[0]
True
>>>
```

The preceding transcript shows storing the integer 10 in the variable `x` and in two distinct one element lists, `lx` and `ly`. In the case of `ly`, the 10 is the result of adding 5 and 5. The `print` function confirms these values. The program then tries comparing the variables using the `is` operator. As you might expect, `x` and `lx` do not reference the same object; one is an integer and the other is a list after all. It's unsurprising that `lx` refers to the same object as `lx` because they are one and the same. Where it gets more interesting is where `lx` and `ly` are shown to be different objects because `lx is ly` evaluates to `False`. When you compare them using the equality operator, you find `lx == ly` to be `True`. Python's equality operator *follows the references* and compares the values they refer to. Because it finds a one-element list stored in both, and the elements in both lists are equal (both are 10), the result is `True`. The `is` operator, by contrast, *does not follow the references*; it just checks whether the references are the same.

If you look at how that data is stored in memory using references, the behavior of `is` and `==` becomes clearer. [Figure 5-2](#) shows the state of the Python interpreter's memory after the assignment statements finish. Each of the three variables is shown with an arrow pointing to its value. There are clearly three copies of the value 10. There are two copies of the list structure. The `is` operator compares where the arrows point and returns `True` if and only if they are identical. The `==` operator walks along the arrows and compares each pair of elements, returning `False` on the first unequal pair or `True` otherwise.

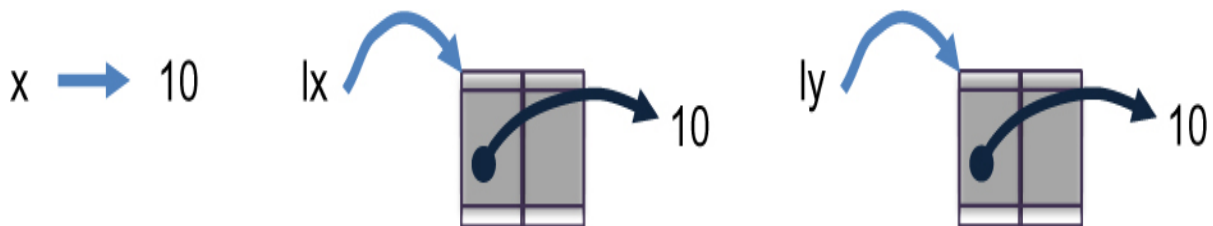


Figure 5-2 *Comparing values in reference data types*

The last line of the preceding transcript compares the first elements of the `lx` and `ly` lists using the `is` operator. They are both 10, so it may not seem

surprising that the operator says they are the same object. It seems reasonable that two primitive data types with the same value compare as being the same using the `is` operator. Be careful, however, because this is a special case in Python. Small integers are stored with the exact same reference in Python, but larger ones are not. For example:

```
>>> 32 == 32
True
>>> 32 is 32
True
>>> 2**32 == 2**32
True
>>> 2**32 is 2**32
False
>>>
```

Big integers can be compared numerically using the equality operator, `==`. When the `is` operator is used, however, the comparison is on the objects created to represent those big integers, and Python creates those big integer objects individually as needed. Big integers don't fit in a machine word or two. This means that it's tricky using arrows to point to integers or other types like in [Figure 5-2](#). What may seem to be a simple primitive might be a more complex structure.

Relationship, Not Position

Let's examine one of the major ways in which linked lists differ from arrays. In an array, each item occupies a particular position. This position can be directly accessed using an index number. It's like a row of houses with sequential addresses: you can find a particular house using its numeric address. If the houses are equally spaced apart and the numbering sequence is equally spaced, you could know exactly how far to travel from the first house in the row to get to a particular numbered house.

In a list, the only way to find a particular element is to follow along the chain of elements. You must visit each of the first elements to get to the one you want. There are two ways you might know which element you want: by its position or by some key. An example of knowing its position is like saying, "It's the fifth house on the left." You don't know exactly how far down the street it is, but you can count the houses as you go and turn left after reaching a count of five. An example of knowing a house by a key would be by giving a

description that distinguishes it from the other houses, for example, the stone house with a green door.

Another example of knowing something by its key is describing someone by their face or name. Imagine that you are at a huge party with hundreds of people. You are looking for an acquaintance, Zev, that you think is attending. Maybe you ask Raj where Zev is. Raj doesn't know, but he thinks Anna might know, so you go and ask Anna. Anna saw Zev go outside with Dimitri. You happen to have Dimitri's phone number, so you call him. He answers but says that Zev went back inside a few minutes ago, saying he was looking for Aoife. So, you start to look for, ... you get the idea. When you start down the chain, you don't even know whether the item you seek is going to be there.

These kinds of chains happen in computer applications too. If you're trying to trace the path a message followed through the network, you start at the node where the message ended (or started) and find a record indicating where the message came from (or went to). You then visit that node and look for the next record showing the message's origin (destination). You must follow all the individual links to reconstruct the full path of the message. These kinds of systems share the property that you can't access a data item directly; you must use relationships between the items to locate a specific item. You start with the first item, go to the second, then the third, until you find what you're looking for, or the end of the relationships.

The Linked List Visualization Tool

Let's look in more detail at how linkages are made in a list data structure. The `LinkedList` Visualization tool shows all the basic operations. You can launch it with the command `python3 LinkedList.py`, following the instructions in [Appendix A, "Running the Visualizations."](#) [Figure 5-3](#) shows what the `LinkedList` Visualization tool looks when it starts. Initially, the list is empty with only a small red circle reserved for where the first link reference will go. The box represents an instance of the `LinkedList` object with its one attribute, as described in [Listing 5-1](#).



Figure 5-3 *The Linked List Visualization tool with an empty list*

To make the first `Link` object, you type a word in the text entry area and select the Insert button. That action creates a colored rectangle holding the word, which represents the `Link` object and its data. The link is the only one in the list, so its pointer is `None`, represented by the red dot. The `LinkedList` object now has a reference to the `Link` stored in its `first` attribute.

Inserting a couple of words using the Insert button could make the start of a shopping list like the one in [Figure 5-4](#). Each `Link` except the last one has a reference to the next `Link`. (The small `link` arrow at the top is left over from the insert operation that just finished.)

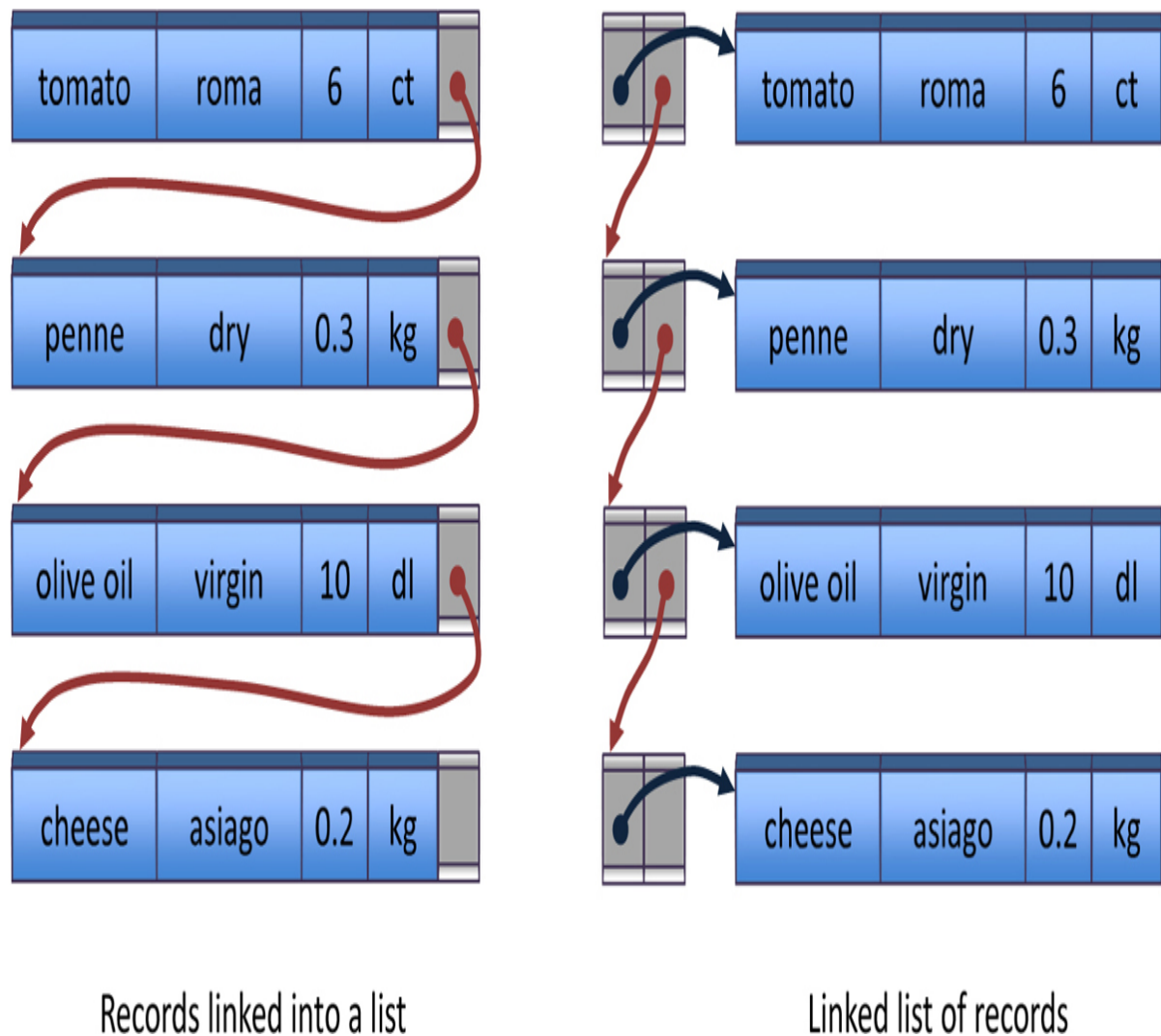


Figure 5-4 *A linked list after inserting some items*

As you insert items, watch the animation closely. The new `Link` begins outside of the list. Its next pointer points to the first item in the list (or nothing when the `LinkedList` is empty). The insertion happens by changing the `first` pointer of the `LinkedList` to point at the new `Link`. The visualization tool then rearranges the linked items to make the familiar chain-like structure. There is no change happening to the data structure in memory when the tool rearranges the items; that just makes the structure easier to see in the visualization.

In this version of a linked list, new links are always inserted at the beginning of the list. This is the simplest approach, although sometimes there are operations that need to place items elsewhere in some variations on the basic list, as we show later.

The Search Button

The Search button allows you to find a link with a specified key value. Type the value of an existing link (or click it) and select Search. The operation begins by placing a `link` pointer at the first `Link`. The `link` pointer advances down the links until it points at a link whose key matches the `goal` key you entered. The object is highlighted, and the data is copied to an output box. If you search for a key that's not in the list, the `link` pointer advances past the last `Link` in the list and ends up pointing at nothing (`None`). The message at the bottom informs you whether the key was found.

The Delete Button

You can also delete a `Link` with a specified key. Type in the value of an existing link and then select Delete. This time, two arrows appear and move along the list, somewhat like an inchworm, looking for the goal link. When the `link` arrow arrives at the link with the `goal` key, the advancing stops, and the previous `Link`'s pointer is reconnected to the `goal` link's successor. After the previous pointer is updated, the `goal` link is no longer in the list and is moved away. If the `goal` link isn't found, the animation stops with an exception.

The visualization tool moves the `Links` around on the display, sometimes stretching and bending the arrows for the pointers. The length and shape of the visualization arrows don't correspond to anything in the program. They only serve to show what objects reference other objects.

The New Button

As with the other data structures, you create a new, empty linked list by selecting the New button. Unlike those other structures, you don't need to specify a size for the structure. The linked list is not based on an array; links are allocated and discarded as needed.

The Other Buttons

The Delete First button deletes the first link in the list, regardless of its key. This is somewhat like the Delete Rightmost operation that you saw with arrays. A Get First button gets the data value stored in the first link, and a Traverse

button walks through all the links in the list, copying their values to an output box.

A Simple Linked List

Let's look at the methods needed to implement the basic operations in the `Link` and `LinkedList` classes in Python.

We expand on the constructors and simple tests shown in [Listing 5-1](#), add methods to access the components and a helper method to display a `Link` as a string in [Listing 5-2](#). This listing introduces `getData()` and `setData()` methods to access the data stored in the `Link`. Another alternative would be to make the `__data` attribute (or field) a public attribute of the `Link` class that calling programs could set. That would allow for direct access to the instance attribute, which might not be desired in some cases, such as enforcing some constraint on the kinds of data that can be added. The `setNext()` method illustrates the idea of a constraint by enforcing that the `__next` attribute can be set to only a reference to a `Link` object or the special “no next” value, `None`. This listing retains the `isLast()` method that defines the convention that a value of `None` in the `__next` field means there are no subsequent links. Lastly, the listing defines a `__str__()` method for showing links as strings. It simply applies the string conversion process to whatever datum is stored in the `Link`.

Listing 5-2 *Basic Methods for Link Objects*

```
class Link(object):
    # One datum in a linked list
    def __init__(self, datum, next=None): # Constructor
        self.__data = datum             # The datum for this link
        self.__next = next               # Reference to next Link

    def getData(self):                   # Return the datum stored in this link
        return self.__data

    def setData(self, datum):           # Change the datum in this Link
        self.__data = datum

    def getNext(self): return self.__next # Return the next link

    def setNext(self, link):            # Change the next link to a new Link
        if link is None or isinstance(link, Link): #Must be Link or None
```

```

        self.__next = link
    else:
        raise Exception("Next link must be Link or None")

def isLast(self):
    # Test if link is last in the chain
    return self.getNext() is None # True if & only if no next Link

def __str__(self):
    # Make a string representation of link
    return str(self.getData())

```

The Basic Linked List Methods

In [Listing 5-3](#) the `LinkedList` class is updated with the same kind of accessor methods for its one attribute, `__first`. Along with the `get` and `set` methods for the first link, it defines `getNext()` and `setNext()` methods as synonyms whose utility will show up a little later. A new method, `first()`, returns the first item in the linked list. It's analogous to the `peek()` method for queues. It raises an exception if the list is empty. Note that to get to the data, it dereferences the `__next` attribute to get a `Link` and then uses that to get that link's data. This is a common operation on linked lists.

Listing 5-3 *Basic Methods for LinkedList Objects*

```

class LinkedList(object):
    # A linked list of data elements
    def __init__(self):
        # Constructor
        self.__first = None # Reference to first Link

    def getFirst(self):
        return self.__first # Return the first link

    def setFirst(self, link):
        # Change the first link to a new Link
        if link is None or isinstance(link, Link): # It must be None or
            self.__first = link # a Link object
        else:
            raise Exception("First link must be Link or None")

    def getNext(self):
        return self.getFirst() # First link is next
    def setNext(self, link):
        self.setFirst(link) # First link is next

    def isEmpty(self):
        # Test for empty list
        return self.getFirst() is None # True iff no first Link

    def first(self):
        # Return the first item in the list

```

```
if self.isEmpty():           # as long as it is not empty
    raise Exception('No first item in empty list')
return self.getFirst().getData() # Return data item (not Link)
```

Traversing Linked Lists

To make the linked list useful, you need to be able to insert, delete, search, and traverse the elements it holds. Traversal is perhaps the easiest and is the essence of helper methods, like getting the length of the list or creating a string representation of its contents. [Listing 5-4](#) shows the traversal methods.

Listing 5-4 *Traversal Methods of `LinkedList`*

```
class LinkedList(object):      # A linked list of data elements

... (basic definitions shown before) ...

    def traverse(self,          # Apply a function to all items in list
                  func=print): # with the default being to print
        link = self.getFirst() # Start with first link
        while link is not None: # Keep going until no more links
            func(link.getData()) # Apply the function to the item
            link = link.getNext() # Move on to next link

    def __len__(self):          # Get length of list
        l = 0
        link = self.getFirst() # Start with first link
        while link is not None: # Keep going until no more links
            l += 1               # Count Link in chain
            link = link.getNext() # Move on to next link
        return l

    def __str__(self):          # Build a string representation
        result = "["           # Enclose list in square brackets
        link = self.getFirst() # Start with first link
        while link is not None: # Keep going until no more links
            if len(result) > 1:  # After first link,
                result += " > " # separate links with right arrowhead
            result += str(link)  # Append string version of link
            link = link.getNext() # Move on to next link
        return result + "]"     # Close with square bracket
```

The `traverse()` method applies a function to each element of the list. It starts by setting a `link` variable to point at the first `Link` in the `LinkedList`. This is a reference to an object. The `while` loop then steps through the list as long as that next `link` is not `None`, which indicates the end of the list. If there is a `Link` object, it applies the desired function (`print` by default) to the data in it and then moves on to that `Link`'s next reference. Note the loop could make use of the `isLast()` method to detect the end of the list. Because it still needs to apply the function to the last `Link`, it's shorter to write the loop by testing the `link` variable.

The `__len__()` method (which enables Python's `len()` function to work on `LinkedList` objects) follows the same outline as `traverse`. It keeps the length of the list in a variable, `l`, and increments that value in the body of the `while` loop. As it sets the `link` to each subsequent `Link` object, the count of the number of objects goes up by 1. When there are no more `Link` objects, it returns the count stored in `l`.

Similarly, the `__str__()` method initializes a string variable called `result` with a left bracket. The `while` loop is the same. On each pass through the loop body, a separator string, `" > "`, is added if the `Link` object is not the first one. Then the string representation of the `Link` is added to the `result`. At the end of the list, a final closing right bracket is added to the `result`, and it is returned.

Insertion and Search in Linked Lists

The insertion and searching methods use similar loops when looking for an item with a matching key in the list, as shown in [Listing 5-5](#). To handle complex data within the linked list, it's convenient to use a function to extract the key from each link's data item. The code uses `identity()` as the default function to get the key for each element of the list.

Listing 5-5 *Insertion and Searching Methods for `LinkedList`*

```
def identity(x): return x          # Identity function

class LinkedList(object):         # A linked list of data elements

... (other definitions shown before) ...
```

```

def insert(self, datum):      # Insert a new datum at start of list
    link = Link(datum,        # Make a new Link for the datum
                 self.getFirst()) # What follows is the current list
    self.setFirst(link)      # Update list to include new Link

def find(                    # Find the 1st Link whose key matches
    self, goal, key=identity): # the goal
    link = self.getFirst()    # Start at first link
    while link is not None:    # Search until the end of the list
        if key(link.getData()) == goal: # Does this Link match?
            return link        # If so, return the Link itself
        link = link.getNext()  # Else, continue on along list

def search(                  # Find 1st item whose key matches goal
    self, goal, key=identity):
    link = self.find(goal, key) # Look for Link object that matches
    if link is not None:        # If found,
        return link.getData()   # return its datum

def insertAfter(             # Insert a new datum after the first
    self, goal, newDatum,    # Link with a matching key
    key=identity):
    link = self.find(goal, key) # Find matching Link object
    if link is None:           # If not found,
        return False           # return failure
    newLink = Link(           # Else build a new Link node with
        newDatum, link.getNext()) # new datum and remainder of list
    link.setNext(newLink)     # and insert after matching link
    return True

```

For inserting values in the list, the simplest thing to do is insert them at the front of the list. Like pushing a value on a stack, the value will become the first item (in the first `Link`) of the list. The `insert()` method does that by creating a new `Link` object whose data is set to the `datum` being inserted. The interesting, and perhaps confusing, step is the initialization of the new `Link` object's `__next` pointer. It gets set to the value of the `LinkedList`'s first pointer. The reason is that the original `LinkedList` has all the elements that are going to come after the newly created `Link`. The final step is to overwrite the `LinkedList`'s `__first` pointer so that it points to the newly created `Link`. The processing steps are illustrated in [Figure 5-5](#).



Figure 5-5 *Execution of `insert()`*

The example in [Figure 5-5](#) shows the ingredient “oil” being added to a list that already contains “eggs” and “milk”. The first panel shows the state of the arguments to the method on entry into `insert("oil")`. The `self` variable is the `LinkedList` to be modified, `l1ist` in this case. The `LinkedList` object has one field, `__first`, that points to the first `Link` in the list. The first `Link` is the one for “eggs” and it’s followed by the `Link` for “milk”. The second panel shows the creation of the new `Link` object stored in the `link` variable. The data is set to the input `datum`, and the `__next` pointer is set to the original `LinkedList`’s `__first` value. The third panel shows how the original `LinkedList`’s `__first` pointer is set to the newly created `Link`. The last panel shows the state right before returning from the call to `insert()` with the `Link` for “oil” clearly positioned at the front of the list, a simple rearrangement of the `Link` structures in the third panel.

The example in [Figure 5-5](#) can also be animated in the Linked List Visualization tool. The figure is a bit more detailed about the state of all the variables than the visualization is. The figure also shows the string data items being referenced through a pointer from each `Link`. The string reference is used in the computer but is difficult to see in the code. Try inserting the same items in a list and using the step button to step through each line of the code for one of the insertions.

If insertion and deletion to the linked list could only occur at the first position in the list, the data structure would behave like a stack where the last element put in is always the first element taken out. To be able to do things like insert an element at a particular position within an ordered list, you need to be able to specify and find positions within the list where the change should be made. For

that, you need to find elements with a particular key, which is what the `find()` method does.

Just like the traversal operations, the `find()` method steps through the `Links` of the list. For each link, the `find()` method shown in [Listing 5-5](#) applies the `key()` function to the data stored there. If the value of the extracted key matches the `goal` key, then the `Link` containing that key is returned. If the `while` loop ends, then none of the `Links` had a matching key, and the result is `None` (because Python returns an implicit `None` when the body of the function finishes). It's important to note that `find()` always returns a `Link` (really a reference to a `Link`) or `None`. It doesn't return an integer index for where the `Link` was found in the list like the `find()` method for arrays. Why not? If it did, then a subsequent operation to insert or delete a `Link` at that position would have to repeat stepping through the links to find the right spot. Returning the reference saves many steps.

To get the data stored at the link with a particular key, you use the `search()` method. This method takes the same arguments as the `find()` method but returns the *data in the Link* rather than a *reference to the Link*. This subtle difference is very important. Some operations need a position, and some need the data at the position. The `search()` method simply calls the `find()` method, and if it returns a pointer to a `Link`, it dereferences the pointer to get the value. Otherwise, it too returns `None`.

The next method in [Listing 5-5](#), `insertAfter()`, provides a way to insert a new datum immediately after the first `Link` in the list that has a key matching a particular `goal` key. It makes use of the `find()` method to discover whether there's a `Link` with a matching key. If none is found, it returns `False` to indicate that. Otherwise, it creates a new `Link` to splice into the list right after the one with the matching key.

Just like the `insert()` method, the new `Link` for `insertAfter()` contains the provided datum, and its `__next` pointer is set to what should follow. In this case, it's the linked list that follows the `Link` with the matching key, as illustrated in [Figure 5-6](#). The figure shows the same datum, "oil", being inserted in a list, but this time it is to be inserted after "eggs". The `link` variable stores the output of the `find()` call. The `newLink` is created in the third panel with "oil" for its data and the list after the "eggs" `Link` as the list that follows. After the `__next` pointer for the insertion `link` is updated to point at the `newLink`, the linked list is altered into its final form shown in the fourth panel. The routine returns `True` to indicate that it found the goal key and

inserted the new `Link`. Note that the links have been rearranged in the bottom right of the figure to better show the new list structure.



Figure 5-6 *Inserting “oil” after “eggs” in a linked list*

Deletion in Linked Lists

The final methods needed for a basic `LinkedList` structure are the ones for deletion. Deleting the first item is another quick operation that simply changes a couple of pointers. Deleting from the middle means finding an item by its key.

The `deleteFirst()` method shown in [Listing 5-6](#) deletes the first link in the list, after checking that the list is not empty. The steps in the code are almost as simple as those for the `insert()` method. It stores a pointer to the first `Link` in the `first` variable. This is necessary to retain access to the data stored there after taking that link out of the list. The second step performs the actual deletion by changing the `LinkedList`’s `__first` attribute to point at the link after the `first` one. The `first` link is now disconnected even though its `__next` pointer points to the remainder of the list. The final step is to return the data from that `first` link.

When you need to delete from somewhere else in the list, the `delete()` method deletes an item with a matching `goal` key. Like the `insertAfter()` and `search()` methods, you first identify the `Link` where the delete should happen and then change the pointers. Although you could use the `find()` method to identify the `Link` to delete, the code shown in [Listing 5-6](#) doesn’t. Why do you think that is?

Listing 5-6 *The deleteFirst() and delete() Methods for LinkedLists*

```
class LinkedList(object):          # A linked list of data elements

... (other definitions shown before) ...

    def deleteFirst(self):          # Delete first Link
        if self.isEmpty():         # Empty list? Raise an exception
            raise Exception("Cannot delete first of empty list")

        first = self.getFirst()    # Store first Link
        self.setFirst(first.getNext()) # Remove first link from list
        return first.getData()     # Return first Link's data

    def delete(self, goal,          # Delete the first Link from the
                  key=identity):   # list whose key matches the goal
        if self.isEmpty():         # Empty list? Raise an exception
            raise Exception("Cannot delete from empty linked list")

        previous = self            # Link or LinkedList before Link
        while previous.getNext() is not None: # to be deleted
            link = previous.getNext() # Next link after previous
            if goal == key(link.getData()): # If next Link matches,
                previous.setNext( # change the previous' next
                                link.getNext()) # to be Link's next and return
                return link.getData() # data since match was found
            previous = link         # Advance previous to next Link

        # Since loop ended without finding item, raise exception
        raise Exception("No item with matching key found in list")
```

The reason for not using the `find()` method is that `delete()` must identify the Link that *precedes* the one to be deleted, not the Link with the matching key itself. By finding the preceding Link, it can modify that link's `__next` pointer to shorten the list and eliminate the desired Link. That means the method uses a different kind of `while` loop where it updates a pointer called `previous` that points at the Link preceding the one with the goal key, if one exists. After checking for an empty list condition and raising an exception if it finds one, the `delete()` method sets `previous` to point at the input `LinkedList (self)`, *not* the first Link in the chain. This is necessary if the Link to be deleted is the first Link in the list. Let's start by looking at that case, in particular.

The `previous` variable initially points to `self`, which is the `LinkedList` to be modified. On a call to the `delete()` method where the `goal` key matches the first `Link` in the linked list, the `while` loop is entered by first checking that the next `Link` after `previous` exists. It makes that check by calling `getNext()`. This explains why `getNext()` was defined as a synonym for `getFirst()`; you want to treat the first pointer of the `LinkedList` just like all the other next pointers in the `Links` that follow.

The `delete()` method's loop sets the `link` variable to point at the `Link` that follows `previous`, which is the first `Link` of the list. When the loop body checks that first `Link`'s key, it matches the `goal` in the first case, so now the method can perform the deletion. The `previous` variable remains pointing at `self`, the `LinkedList` to be modified. By setting the next link of `previous` using `setNext()`, the `delete()` method modifies the input `LinkedList`'s `__first` link. This is the other synonym method defined in [Listing 5-3](#) so that you can apply `setNext()` to modify either the `__first` attribute or the `__next` attribute as appropriate. It sets `__first` to be the `Link` following the one whose key matched the `goal`. That means the `Link` with the matching key has been removed from the list. Then the method can return the link's data because it found the `Link` to delete.

The second case to examine is what happens when the first `Link` in the list doesn't match the `goal` key to be deleted. [Figure 5-7](#) illustrates the two cases, showing where `previous` is pointing right before the call to `previous.setNext()`. The `previous` and `link` variables start off the same in the second case, but this time, the first `Link`'s key doesn't match the `goal`. It skips the body of the inner `if` statement and sets `previous` to the value of `link`, which is the next `Link` in the list. Now `previous` is the first `Link` in the list, and the `while` loop checks whether there are any more `Links` after it. If there are more, the `link` variable is assigned to the next one, and its key is checked against the `goal`. When the key of `link` matches, the `previous` variable will point to the `Link` that immediately precedes `link`. The body of the `if` statement removes the `goal` `Link` in the same way as if it were the first `Link` in the list, thanks to the `setNext()` method working on both `LinkedLists` and `Links`.



Figure 5-7 *Two examples of deleting a `Link` from a `LinkedList`*

In the first case we discussed, `previous` points to an object of type `LinkedList`, and in the second case it points to a `Link` object. We designed both classes to have `getNext()` and `setNext()` methods to make this kind of while loop in `delete()` easier to write. This is an example of **polymorphism** in object-oriented programming—where the same operation can be performed on different object types with each type (class) having its own implementation. The idea is to treat both the `__first` and `__next` pointers in the same way across both classes. The reference that is about to be set is shown as a dashed arrow in [Figure 5-7](#).

Double-Ended Lists

A double-ended list is like an ordinary linked list, but it has one additional feature: a reference to the `__last` link as well as to the first. [Figure 5-8](#) shows such a list.

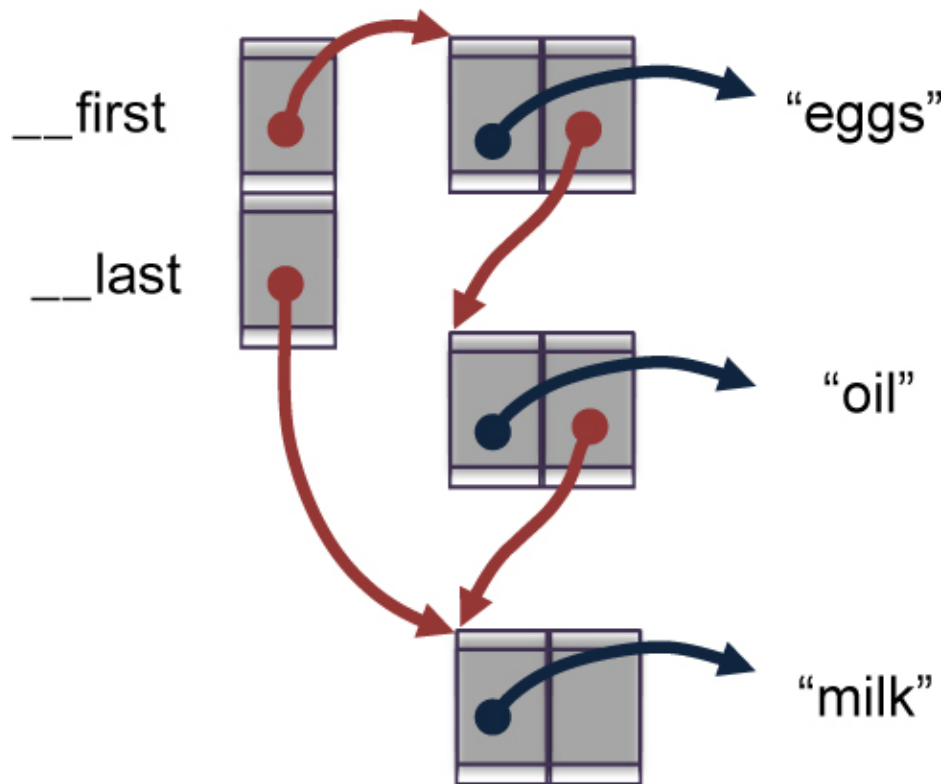


Figure 5-8 *A double-ended list*

The reference to the `___last` link permits inserting new links directly at the end of the list as well as at the beginning. Of course, you can insert a new link at the end of an ordinary single-ended list by iterating through the entire list until you reach the end. That approach, however, becomes quite inefficient when the list is long.

Quick access to the end of the list as well as the beginning makes the double-ended list suitable for certain situations that a single-ended list can't handle efficiently. One such situation is implementing a queue; we show how this technique works in the next section. There is a small cost for that benefit—maintaining the pointer to the other end—as shown in [Listing 5-7](#).

Listing 5-7 *The `DoubleEndedList` Class*

```
from LinkedList import *

class DoubleEndedList(LinkedList): # A linked list with access to both
    def __init__(self):           # ends of the list
        self.__first = None       # Reference to first Link, if any
```

```

self.__last = None          # Reference to last link, if any

def getFirst(self): return self.__first # Return the first link

def setFirst(self, link):    # Change the first link to a new Link
    if link is None or isinstance(link, Link): #Must be Link or None
        self.__first = link    # Update first link
        if (link is None or      # When removing the first Link or
            self.getLast() is None): # the last Link is not set,
            self.__last = link # then update the last link, too.
    else:
        raise Exception("First link must be Link or None")

def getLast(self): return self.__last # Return the last link

def last(self):          # Return the last item in the list
    if self.isEmpty():   # as long as it is not empty
        raise Exception('No last element in empty list')
    return self.__last.getData()

def insertLast(self, datum): # Insert a new datum at end of list
    if self.isEmpty():       # For empty lists, end is the front,
        return self.insert(datum) # so insert there
    link = Link(datum, None) # Else make a new end Link with datum
    self.__last.setNext(link) # Add new Link after current last
    self.__last = link        # Change last to new end Link

def insertAfter(          # Insert a new datum after the 1st
    self, goal, newDatum, # Link with a matching key
    key=identity):
    link = self.find(goal, key) # Find matching Link object
    if link is None:           # If not found,
        return False          # return failure
    newLink = Link(            # Else build a new Link node with
        newDatum, link.getNext()) # new datum and remainder of list
    link.setNext(newLink)      # and insert after matching link
    if link is self.__last:    # If the update was after the last,
        self.__last = newLink # then update reference to last
    return True

def delete(self, goal,      # Delete the first Link from the
    key=identity):         # list whose key matches the goal
    if self.isEmpty():      # Empty list? Raise an exception
        raise Exception("Cannot delete from empty linked list")

    previous = self         # Link or LinkedList before Link
    while previous.getNext() is not None: # to be deleted

```

```

link = previous.getNext() # Next link after previous
if goal == key(link.getData()): # If next Link matches,
    if link is self.__last: # and if it was the last Link,
        self.__last = previous # then move last back 1
    previous.setNext( # Change the previous' next
        link.getNext()) # to be Link's next and return
    return link.getData() # data since match was found
previous = link # Advance previous to next Link

# Since loop ended without finding item exception
raise Exception("No item with matching key found in list")

```

We choose to implement a double-ended list by making a subclass of the `LinkedList` that we've been studying. That's done in the `class` statement, which shows the `DoubleEndedList` inheriting the definition of `LinkedList`. That means we only need to redefine methods that change from those that were used in the parent, or "super," class. The constructor method, `__init__()`, differs only in that it sets the `__last` pointer to initially be empty, like the `__first` pointer.

The next definitions in [Listing 5-7](#) are the accessor methods for the first link of the list. If you look carefully, you'll notice that the `getFirst()` method is identical to the definition in the parent class, `LinkedList`. That seems strange because subclasses only need to redefine methods that have changed from the definition in their super classes. The reason for redefining it is that Python has a special way of handling private class fields like `__first` and `__last`. Python treats those variables as being completely private to the class where they are defined. If subclasses use private fields with the same name, they are distinct fields from those of the parent class. So, the definition of the `getFirst()` method is really making an accessor for the `__first` variable of the `DoubleEndedList` class in [Listing 5-7](#), and that is different from the `LinkedList.getFirst()` method. The new definition replaces that of the superclass so that all calls to `getFirst()` will get the `DoubleEndedList` `__first` field.

The differences between the `DoubleEndedList` class and its parent begin to show up in the `setFirst()` method. Like the definition in the super class, it verifies that the new value is either an instance of the `Link` class or `None`. After setting the `__first` field, it goes on to see whether an update to the `__last` field is needed. The last `Link` needs to be updated when the first link is added to an empty list, or when the last `Link` is being removed from the list. The `if` statement checks for the removal of the last link by testing whether the new

`link` is `None`. It checks for the addition of a potential new `Link` by testing whether the `__last` field is `None`. Note that it checks the status of the `__last` field by using the accessor method, `getLast()`, which is defined later. Avoiding references to the private attributes enables future subclassing of this class.

The `DoubleEndedList` class defines a `getLast()` method but not a `setLast()` method. That's intentional because it is not wise to allow callers to change the internal, private field in a way that could point that field somewhere other than the last `Link` of the list. All changes should be managed by the code in the class definition. (To further protect the `DoubleEndedList` class from corruption, the `setFirst()`, method would need to be rewritten to ensure that the `__first` and `__last` attributes always point to the beginning and end of the same linked list.)

The definition also includes a `last()` method, which, like the `first()` method, gets the indicated data rather than a reference to the `Link` object holding the data. The `first()` method is inherited from the `LinkedList` class, as are the synonym definitions for `getNext()` and `setNext()`.

Continuing in [Listing 5-7](#), you find the new `insertLast()` method, which exploits the presence of the `__last` field to make adding a `Link` at the end of the list very efficient. If the list is empty, then inserting at the end is the same as inserting at the front of the list. Otherwise, a new terminal `Link` object is created. The logic to insert it is the same as that used in `insert()`, except that the `__next` field of the last `Link` is updated rather than the `__first` field of the `DoubleEndedList`. Because a new `Link` is added, the `__last` field must also be updated too.

The last two methods of the `DoubleEndedList` class are slight rewrites of the same methods in `LinkedList`. The `__last` field must be updated appropriately after insertions and deletions. In the `insertAfter()` method, a new `if` statement is added at the end to check if the `link` that it modifies is the last `Link` of the list. If it is the last one, then the `__last` field must be advanced to point at the newly added `Link`. In the `delete()` method, another new `if` statement is added just before modifying the previous `Link`'s `__next` (or `LinkedList`'s `__first`) pointer. If the link to delete is the same as the last link, then the `__last` field must be updated to point at the previous link.

There is one special case to consider during deletion: when `previous` points to the `DoubleEndedList` and not a `Link` object, the final link is being deleted. By

changing `__last` to be `previous` before calling `previous.setNext()`, the test in `setNext()/setFirst()` will find that the new value for `__first` is `None` and set both `__first` and `__last` of the `DoubleEndedList` to be `None`.

Interestingly, the `DoubleEndedList` does not need to redefine the `deleteFirst()` method inherited from `LinkedList`. When deleting the first link, the `deleteFirst()` method calls `setFirst()`, which has already been customized in the `DoubleEndedList` to update its private `__last` pointer.

You can exercise the features of the `DoubleEndedList` class using the program shown in [Listing 5-8](#). The client program is a little fancier by using records for the items instead of just strings. It builds an empty double-ended list, `dlist`, and then inserts pairs of numbers with names into the list. It uses the second element of each pair, the name, as the key for the data. The numbers in the record help indicate the order that they were inserted. It puts several people into the list, inserting the first one at the beginning, setting a variable called `after` to the person in that first datum, and then inserting all the other pairs after the first.

Listing 5-8 *The `DoubleEndedListClient.py` Program*

```
from DoubleEndedList import *

def second(x): return x[1]

dlist = DoubleEndedList()

print('Initial list has', len(dlist), 'element(s) and empty =',
      dlist.isEmpty())
after = None
people = ['Raj', 'Amir', 'Adi', 'Don', 'Ken', 'Ivan']
for i, person in enumerate(people):
    if after:
        dlist.insertAfter(after, (i * i, person), key=second)
    else:
        dlist.insert((i * i, person))
        after = person

print('After inserting', len(dlist) - 1,
      'persons into the linked list after,', after, 'it contains:')
dlist.traverse()
print('First:', dlist.first(), 'and Last:', dlist.last())
```

```

next = (404, 'Tim')
dlist.insertLast(next)
print('After inserting', next, 'at the end, the double-ended list',
      'contains:\n', dlist)

dlist.insert(next)
print('After inserting', next, 'at the front, the double-ended list',
      'contains:\n', dlist)
print('Deleting the first item returns', dlist.deleteFirst(),
      'and leaves the double-ended list containing:\n', dlist,
      'with first:', dlist.first(), 'and Last:', dlist.last())
print('Deleting the last item returns',
      dlist.delete(second(dlist.last()), key=second),
      'and leaves the double-ended list containing:\n', dlist,
      'with first:', dlist.first(), 'and Last:', dlist.last())

print('Removing some items from the linked list by key:')
for person in people[0:5:2]:
    dlist.delete(person, key=second)
    print('After deleting', person, 'the list is', dlist)
    if not dlist.isEmpty():
        print('The last item is', dlist.last())

print('Removing remaining items from the front of the linked list:')
while not dlist.isEmpty():
    print('After deleting', dlist.deleteFirst(), 'the list is', dlist)
    if not dlist.isEmpty():
        print('The last item is', dlist.last())

```

After the initial data is inserted, it displays the contents of the list using the `traverse()` method. The specific first and last data are also printed. The program adds another pair, `next`, at the end and beginning of the list, printing the contents of the list using the string conversion capability after each insertion. The next test deletes those first and last items, exercising the `deleteFirst()` and `delete()` methods.

Finally, it removes some items based on their keys, using a few of the `people` names. Then it removes the rest by deleting the first item in the list. After each deletion, it shows the content of the list and the last data item. The output of the program is

```

$ python3 DoubleEndedListClient.py
Initial list has 0 element(s) and empty = True
After inserting 5 persons into the linked list after, Raj it contains:

```

```

(0, 'Raj')
(25, 'Ivan')
(16, 'Ken')
(9, 'Don')
(4, 'Adi')
(1, 'Amir')
First: (0, 'Raj') and Last: (1, 'Amir')
After inserting (404, 'Tim') at the end, the double-ended list
contains:
[(0, 'Raj') > (25, 'Ivan') > (16, 'Ken') > (9, 'Don') > (4, 'Adi') >
(1,
'Amir') > (404, 'Tim')]
After inserting (404, 'Tim') at the front, the double-ended list
contains:
[(404, 'Tim') > (0, 'Raj') > (25, 'Ivan') > (16, 'Ken') > (9, 'Don')
>
(4, 'Adi') > (1, 'Amir') > (404, 'Tim')]
Deleting the first item returns (404, 'Tim') and leaves the double-
ended
list containing:
[(0, 'Raj') > (25, 'Ivan') > (16, 'Ken') > (9, 'Don') > (4, 'Adi') >
(1,
'Amir') > (404, 'Tim')] with first: (0, 'Raj') and Last: (404, 'Tim')
Deleting the last item returns (404, 'Tim') and leaves the double-
ended
list containing:
[(0, 'Raj') > (25, 'Ivan') > (16, 'Ken') > (9, 'Don') > (4, 'Adi') >
(1,
'Amir')] with first: (0, 'Raj') and Last: (1, 'Amir')
Removing some items from the linked list by key:
After deleting Raj the list is [(25, 'Ivan') > (16, 'Ken') > (9,
'Don') >
(4, 'Adi') > (1, 'Amir')]
The last item is (1, 'Amir')
After deleting Adi the list is [(25, 'Ivan') > (16, 'Ken') > (9,
'Don') >
(1, 'Amir')]
The last item is (1, 'Amir')
After deleting Ken the list is [(25, 'Ivan') > (9, 'Don') > (1,
'Amir')]
The last item is (1, 'Amir')
Removing remaining items from the front of the linked list:
After deleting (25, 'Ivan') the list is [(9, 'Don') > (1, 'Amir')]
The last item is (1, 'Amir')
After deleting (9, 'Don') the list is [(1, 'Amir')]
The last item is (1, 'Amir')
After deleting (1, 'Amir') the list is []

```

The initial pair, `(0, 'Raj')`, stays at the beginning of the list while the other pairs are inserted after it. Their numeric values decrease, which shows that their order in the completed list is the *reverse* of the order they were inserted. This is not an ordered list. The insertion position is defined by the goal key passed to `insertAfter()`. The insertion of `(404, 'Tim')` is done with both `insertLast()` and `insert()`, so it doesn't even look at the keys of the list items. As the items are deleted, the program prints the list and the last element, just to verify that the `__last` pointer is being updated correctly.

The `DoubleEndedList` makes it easy to insert at both the front and end of the list as well as after a `Link` having a particular key. The `delete()` method always looks for a particular key, and it's easy to delete the first and last keys by using the `first()` and `last()` methods. Unfortunately, deleting the last link is still time-consuming because the program still must walk through the `Links` to find the last key. To make it fast, you could add a reference to the next-to-last link, whose `__next` field would need to be changed to `None` when the last link is deleted. You already saw what it takes to modify the `LinkedList` to become the `DoubleEndedList`. Adding more and more fields like `second_to_last`, `third_to_last`, and so on, would be a lot of work, and it's unclear how much more efficiency that would achieve because you could have lists of arbitrary length. To conveniently delete the last link, a better approach is a *doubly linked* list, which we look at soon.

Linked List Efficiency

Insertion and deletion at the beginning of a linked list are very fast. They involve changing only one or two references, which takes $O(1)$ time.

Finding, deleting, or inserting next to a specific item requires searching through, on average, half the items in the list. This operation requires $O(N)$ comparisons. Arrays have the same complexity for these operations, $O(N)$, but the linked list is faster because nothing needs to be copied when an item is inserted or deleted. The increased efficiency can be significant, especially if a copy takes much longer than a comparison.

Of course, another important advantage of linked lists over arrays is that a linked list uses exactly as much memory as it needs and can expand to fill all available memory. The size of an array is fixed when it's created; this could lead to inefficiency when the initial array size is too large, or it could lead to running out of room because the array is too small. Expandable arrays may

solve this problem to some extent, but they usually expand in fixed-sized increments (such as doubling the size of the array whenever it's about to overflow). Typically, the data must be copied from the smaller to larger array when they expand. This adaptive solution is still not as efficient a use of memory as a linked list.

Abstract Data Types and Objects

In this section we shift gears and discuss a topic that's more general than linked lists: abstract data types (ADTs). What is an ADT? Is it the same as an object? Roughly speaking, an ADT is a way of looking at a data structure—focusing on what it does and ignoring how it does its job. Objects are a way of making concrete implementations of ADTs.

Stacks and queues are examples of ADTs. You've already seen that both stacks and queues can be implemented using arrays. Before we return to a discussion of ADTs, let's see how stacks and queues can be implemented using linked lists. This discussion demonstrates the “abstract” nature of stacks and queues—how they can be considered separately from their implementation.

A Stack Implemented by a Linked List

The implementation of the stack in [Chapter 4](#), “[Stacks and Queues](#),” used an ordinary array to hold the stack's data. The stack's `push()` and `pop()` operations were carried out by array operations such as

```
a[top] = item
```

and

```
top = a[j]
```

which put data into and returned it from an array.

You can also use a linked list to hold a stack's data. In this case the `push()` and `pop()` operations could be carried out by operations like

```
theList.insert(data)
```

and

```
data = theList.deleteFirst()
```

The user of the stack class calls `push()` and `pop()` to insert and delete items without knowing, or needing to know, whether the stack is implemented as an array or as a linked list. [Listing 5-9](#) shows two ways of how a stack class can be implemented using the `LinkedList` class instead of an array.

Listing 5-9 *The LinkStack.py Module*

```
from LinkedList import *

class LinkStack(object):
    def __init__(self):                # Constructor for a
        self.__sList = LinkedList()   # stack stored as a linked list

    def push(self, item):              # Insert item at top of stack
        self.__sList.insert(item)     # Store item

    def pop(self):                    # Remove top item from stack
        return self.__sList.deleteFirst() # Return first and delete it

    def peek(self):                  # Return top item
        if not self.__sList.isEmpty(): # If stack is not empty
            return self.__sList.first() # Return the top item

    def isEmpty(self):               # Check if stack is empty
        return self.__sList.isEmpty()

    def __len__(self):               # Return # of items on stack
        return len(self.__sList)

    def __str__(self):               # Convert stack to string
        return str(self.__sList)

class Stack(LinkedList):             # Define stack by renaming
    push = LinkedList.insert        # Push is done by insert
    pop = LinkedList.deleteFirst    # Pop is done by deleteFirst
    peek = LinkedList.first         # Peek is done by first
```

The first thing to note about the `LinkStack` and `Stack` class definitions is how short they are. The implementation of a stack using a linked list is straightforward because they share so many common features. In `LinkStack`, the constructor allocates an empty linked list, `__sList`, and the other methods are basically translations of the common list methods. In the case of a stack, the key function for the `LinkedList` find, search, and delete functions is never

used because stacks don't need keyed access to elements. Another difference is that the constructor doesn't require a maximum size parameter like the `SimpleStack` class does (in [Chapter 4](#)). The reason is that the implementation is not using an array that needs an initial size.

The `Stack` class definition is only four lines in Python. Instead of creating a `LinkedList` object as an internal attribute, it is defined as a subclass of `LinkedList`, inheriting all its attributes and methods. All that is needed is to provide the translations for those method names that differ between a stack and a list.

The shorthand definition of the `Stack` class might seem incomplete to new Python programmers. Normally, there would be some `def` statements for its methods. Here you simply define some class attributes: `push`, `pop`, and `peek`. This approach takes advantage of the way that Python represents class methods as class attributes that are bound to functions instead of some other data type. The statement `push = LinkedList.insert` takes the `insert` attribute (method) of the `LinkedList` class and assigns it to the `push` attribute of the `Stack` class. Because that attribute is executable (Python treats it as *callable*), the attribute becomes a method. The three assignment statements take care of all the changed method names. The remaining methods like `isEmpty()` and the constructor are inherited from `LinkedList`.

To see how both stack implementations work, it's a good idea to test them with something like the `LinkStackClient.py` program shown in [Listing 5-10](#). The outer loop creates two empty stacks, one of each kind, and sets the `stack` variable to each one in turn. The first statement in the loop body prints the data type, shows the stack's string representation, and tests its `isEmpty()` method. The inner `for` loop pushes some squares on the stack. That's followed by printing the full stack contents and tests of the `len()` and `peek()` methods. The final `while` loop then pops the items off, one by one, showing the stack contents and length after each operation.

Listing 5-10 *The `LinkStackClient.py` Program*

```
from LinkStack import *

for stack in (LinkStack(), Stack()):
    print('\nInitial stack of type', type(stack),
          'holds:', stack, 'is empty =', stack.isEmpty())
```

```

for i in range(5):
    stack.push(i ** 2)

print('After pushing', len(stack),
      'squares on to the stack, it contains', stack)
print('The top of the stack is', stack.peek())

while not stack.isEmpty():
    print('Popping', stack.pop(), 'off of the stack leaves',
          len(stack), 'item(s):', stack)

```

The results of running `LinkStackClient.py` are

```
$ python3 LinkStackClient.py
```

```

Initial stack of type <class 'LinkStack.LinkStack'> holds: [] is empty =
True
After pushing 5 squares on to the stack, it contains [16 > 9 > 4 > 1 >
0]
The top of the stack is 16
Popping 16 off of the stack leaves 4 item(s): [9 > 4 > 1 > 0]
Popping 9 off of the stack leaves 3 item(s): [4 > 1 > 0]
Popping 4 off of the stack leaves 2 item(s): [1 > 0]
Popping 1 off of the stack leaves 1 item(s): [0]
Popping 0 off of the stack leaves 0 item(s): []

Initial stack of type <class 'LinkStack.Stack'> holds: [] is empty =
True
After pushing 5 squares on to the stack, it contains [16 > 9 > 4 > 1 >
0]
The top of the stack is 16
Popping 16 off of the stack leaves 4 item(s): [9 > 4 > 1 > 0]
Popping 9 off of the stack leaves 3 item(s): [4 > 1 > 0]
Popping 4 off of the stack leaves 2 item(s): [1 > 0]
Popping 1 off of the stack leaves 1 item(s): [0]
Popping 0 off of the stack leaves 0 item(s): []

```

Note that the `LinkStackClient.py` program doesn't need to know how the `LinkStack` or `Stack` classes are implemented. If you also imported the `SimpleStack` module of [Chapter 4](#) and added a call to the `SimpleStack.Stack(10)` class constructor (where a maximum stack size is required) to the values for `stack` in the outer loop, the rest of the `LinkStackClient.py` program would work identically on that structure. To the

programmer writing `LinkStackClient.py`, there is no difference between using the list-based `LinkStack` class and using the array-based `Stack` class from the `SimpleStack` module of [Chapter 4](#), except for providing a maximum stack size and seeing a different string format for the stack contents.

A Queue Implemented by a Linked List

Here's a similar example of an ADT implemented with a linked list. [Listing 5-11](#) shows a queue implemented as a double-ended linked list.

Listing 5-11 *The `LinkQueue.py` Module*

```
from DoubleEndedList import *

class Queue(DoubleEndedList):          # Define queue by renaming
    enqueue = DoubleEndedList.insertLast # Enqueue/insert at end
    dequeue = DoubleEndedList.deleteFirst # Dequeue/remove at first
    peek = DoubleEndedList.first       # Front of queue is first
```

Similar to the stack, the queue ADT doesn't need to access elements of the queue based on a key, only by their position. It defines the method names needed for a queue using the counterpart methods of the `DoubleEndedList`. The very short 3-line definition looks very simple, but it somewhat hides a fundamental name conflict.

The `Queue` class you saw in [Chapter 4](#) used the `insert()` and `remove()` methods for insertion and removal. If those same method names are used for the `Queue` class based on the `DoubleEndedList` shown in [Listing 5-11](#), a problem would arise although it's hard to see. If you review the definition of `insertLast()` in [Listing 5-7](#), it calls `insert()` when the list is empty to invoke the `LinkedList.insert()` method. If the new `Queue` class definition redefines the `insert()` method, however, then those calls would be redirected to the `DoubleEndedList.insertLast()` method. That method would call the `insert()` method again for an empty list and thus cause an infinite loop. A simple way to avoid that problem is to use the `enqueue()` and `dequeue()` method names for the queue's primary operations.

You can use a similar client program to test the new implementation of the queue with its `enqueue()` and `dequeue()` methods, as shown in [Listing 5-12](#).

Listing 5-12 *The LinkQueueClient.py Program*

```
from LinkQueue import *

queue = Queue()

print('Initial queue:', queue, 'is empty =', queue.isEmpty())

for i in range(5):
    queue.enqueue(i ** 2)

print('After inserting', len(queue),
      'squares on to the queue, it contains', queue)
print('The front of the queue is', queue.peek())

while not queue.isEmpty():
    print('Removing', queue.dequeue(), 'off of the queue leaves',
          len(queue), 'item(s):', queue)
```

The program creates a queue using the `Queue()` constructor; shows its initial state, which is empty; inserts five squares; displays the queue contents; peeks at the front, and then removes items from the queue, displaying the contents after each removal. Here's the output:

```
$ python3 LinkQueueClient.py
Initial queue: [] is empty = True
After inserting 5 squares on to the queue, it contains [0 > 1 > 4 > 9
>
16]
The front of the queue is 0
Removing 0 off of the queue leaves 4 item(s): [1 > 4 > 9 > 16]
Removing 1 off of the queue leaves 3 item(s): [4 > 9 > 16]
Removing 4 off of the queue leaves 2 item(s): [9 > 16]
Removing 9 off of the queue leaves 1 item(s): [16]
Removing 16 off of the queue leaves 0 item(s): []
```

Here, the methods `enqueue()` and `dequeue()` in the `Queue` class are implemented by the `insertLast()` and `deleteFirst()` methods of the `DoubleEndedList` class. Compare this to the array used to implement the queue in the `Queue.py` module of [Chapter 4](#) and see how much shorter the definition is.

The `LinkStack.py` and `LinkQueue.py` modules emphasize that stacks and queues are conceptual entities, separate from their implementations. A stack can be implemented equally well by an array or by a linked list. What's important about a stack is its `push()`, `pop()`, and `peek()` operations and how they're used, not the underlying mechanism used to implement them.

When would you use a linked list as opposed to an array as the implementation of a stack or queue? One consideration is how accurately you can predict the amount of data the stack or queue will need to hold. If this isn't clear, the linked list gives you more flexibility than an array. Both are fast, so speed is probably not a major consideration.

Data Types and Abstraction

Where does the term *abstract data type* come from? Let's look at the *data type* part of it first and then return to *abstract*.

Data Types

The term *data type* can be used in many ways. It is often used to describe built-in types such as `int`, `float`, and `str` in Python or equivalent types in other programming languages. This might be what you first think of when you hear the term.

When you talk about a primitive type, you're actually referring to two things: a data item represented as a collection of bits with certain characteristics, and the permissible operations on that data. For example, a Boolean type variable in Python can have two values—`True` or `False`—and the operators `not`, `and`, and `or` can be applied to them. An `int` variable has whole-number values, and operators like `+`, `-`, `*`, and `/` can be applied to them. The data type's permissible operations are an inseparable part of its identity; understanding the type means understanding what operations can be performed on it.

Object-oriented programming allows programmers to create new data types using classes. Some of these data types represent numerical quantities that are used in ways similar to primitive types. You can, for example, define a class for time duration (with fields for hours, minutes, seconds), and a class for fractions (with numerator and denominator fields). All these classes can be added, subtracted, multiplied, and divided like `int` and `float`. Typically, the new object methods are defined with functional notation like `add()`, `sub()`, `mul()`,

and `div()`. That leads to expressions like `a.sub(b).mul(c)` that don't look like the math expressions we teach and use, but they still produce valid results. Python and some other languages have ways that programmers can define methods so that built-in operators like `+` and `-` can work with them. For example, if you define a method named `__add__(a, b)` for a class in Python, it can be used with the built-in `+` operator. These mechanisms give programmers control over defining all the permissible operations of a data type.

The phrase *data type* seems to fit naturally with quantity-oriented classes. It can, however, apply to classes that don't have this quantitative aspect. In fact, *any* class represents a data type, in the sense that a class is made up of data (fields) and permissible operations on that data (methods). They are not, however, one and the same. A class goes beyond defining a data type or types and the permissible operations that can be performed on them because it also supplies the implementation details.

When a data storage structure like a stack or queue is represented by a class, it too can be referred to as a “data type.” That might sound odd at first, to someone just learning about data storage structures. A stack is different in many ways from an `int`, but they are both defined as a certain arrangement of data and a set of operations on that data. When that new structure is not part of the language or its standard libraries, it's usually not called a “primitive” nor a “built-in” data type. Instead, it might be called a “user-defined” or “program-defined” data type, but it is still a data type.

Abstraction

The word *abstract* means “considered apart from detailed specifications or implementation.” An abstraction is the essence or important characteristics of something. For example, refrigeration is an abstraction. Its important characteristic is keeping something or someplace colder than it would normally be. It is distinct from a refrigerator, which is a device that makes things colder. The legislature of a state or country is another example. The legislature can be considered apart from the individual legislators who happen to perform that duty. The powers and responsibilities of the office remain roughly the same over time while individual officeholders come and go much more frequently.

In object-oriented programming, an **abstract data type** is a class considered without regard to its implementation. It's a description of the data in the class (fields or attributes), the relationships of the fields, a list of operations

(methods) that can be carried out on that data, and instructions on how to use these operations. Specifically excluded are the details of how the methods carry out their tasks. As a class user, you're told what the fields mean, what values they can take, what methods to call, how to call them, and the results you can expect, but not how they work.

The meaning of *abstract data type* is further extended when it's applied to data structures such as stacks and queues. As with any class, it means the data and the operations that can be performed on it, but in this context even the fundamentals of how the data is stored become invisible to the user. Users not only don't know how the methods work; they also don't know what structure(s) store the data. Although the exact mechanism isn't known, users usually do know the complexity of the methods, that is, whether they are $O(1)$, $O(\log N)$, $O(N)$, and so on. When the insertion into a stack or a queue is $O(1)$, you expect all implementations regardless of their implementation to maintain that efficiency.

For the stack, the user (programmer) knows that `push()`, `pop()`, and `peek()` (and perhaps a few other methods like `isEmpty()`) exist and what they are supposed to do. The user doesn't (at least not usually) need to know how `push()` and `pop()` work, or whether data is stored in an array, a linked list, or some other data structure like a tree. In this book, we usually refer to the abstract data type, like an array, without capitalization or a special font. For a class like `LinkStack`, its methods, or a particular instance or object of that class, we use capitalization and the font to distinguish it from the abstract data type.

The Interface

An ADT specification is often called an **interface**. It's what the class user sees—usually its public methods and fields. In a stack class, `push()` and `pop()` and similar methods form the interface. Public attributes like `nItems` are also part of the interface.

ADT Lists

Now that you know what an abstract data type is, here's another one: the *list*. A list (sometimes called a linear list) is a group of items arranged in a linear order. That is, they're lined up in a certain way, like beads on a string or links in a chain. Lists support certain fundamental operations. You can insert an

item, delete an item, and usually read an item from a specified location (the first, the last, and perhaps an intermediate item specified by a key or an index). The exact choice of which of those operations is possible is part of the definition of the ADT.

Don't confuse the ADT list with the linked list classes discussed in this chapter, `LinkedList` and `DoubleEndedList`, nor Python's `list` data type. An ADT list is defined by its interface—the specific methods and attributes used to interact with it. This interface can be implemented by various structures, including arrays and linked lists. The list is an abstraction of such data structures. The classes are objects, and objects are a way of implementing abstract data types. When detailed information is added about storage and algorithms, it becomes a **concrete data type**.

ADTs as a Design Tool

The ADT concept is a useful aid in the software design process. If you need to store data, start by considering the operations that need to be performed on that data. Do you need access to the last item inserted? The first one? An item with a specified key? An item in a certain position? How frequently do you expect each kind of access to occur? Answering such questions leads to the choice of an appropriate ADT or the definition of new one. Only after the ADT is completely defined should you worry about the details of how to represent the data and how to code the methods that access the data.

By decoupling the specification of the ADT from the implementation details, you can simplify the design process. You also make it easier to change the implementation at some future time. If a programmer writes a design or even develops some code using only the ADT interface, a second programmer should be able to change the implementation of the ADT without “breaking” the first programmer's code.

Of course, after designing the ADT, the underlying data structure and algorithm implementation must be carefully chosen to make the specified operations as efficient as possible. If you need random access to the *N*th element, for example, the linked-list representation isn't so good because random access isn't an efficient operation for a linked list. You'd be better off with an array. If random access is needed only very rarely, and the extra time needed to find the item is small compared to the time it could take to copy the

array contents several times when growing (and maybe shrinking) an expandable array, then maybe the linked list representation is better.

Note

Remember that the ADT concept is only a conceptual tool. Data storage structures are not divided cleanly into some that are ADTs and some that are used to implement ADTs. A linked list, for example, doesn't need to be wrapped in a list interface to be useful; it can act as an ADT on its own, or it can be used to implement another data type such as a queue. A linked list can be implemented using an array, and an array-type structure can be implemented using a linked list. In fact, Python's `list` data type is implemented using arrays and acts like both an array and a list. This implementation is “invisible” to most programmers, although it's important to know that adding an element to the end of Python `list` is usually an $O(1)$ operation, not $O(N)$.

Ordered Lists

In the linked lists you've seen thus far, there was no requirement that data be stored in order. For many applications, however, it's useful to maintain the data in a particular order within the list. A list with this characteristic is called an **ordered list**.

In an ordered list, the items are always arranged in order by a key value. Deletion is often limited to the smallest (or the largest) item in the list, which is at the start of the list, although sometimes `find()` and `delete()` methods, which search through the list for specified links, are used as well.

In general, you can use an ordered list in most situations in which you use an ordered array. The advantages of an ordered list over an ordered array are speed of insertion (because elements don't need to be moved) and the fact that a list can expand to meet the minimum memory required, whereas an array is limited to a fixed size that must be declared before using it. An ordered list may, however, be more difficult to implement than an ordered array.

Later we look at one application for ordered lists: sorting data. An ordered list can also be used to implement a priority queue, although a heap (see [Chapter 13](#), “**Heaps**”) is a more common implementation. Note that it is very rare to start with an unordered linked list and sort its contents. Doing so efficiently takes extra work. We use the term *ordered list*, as opposed to *sorted list*, to reinforce that the data is always kept in order, not mixed up and later rearranged.

The Linked List Visualization tool introduced at the beginning of this chapter demonstrated unordered lists. To see how ordered lists work, use the `OrderedList` Visualization tool, launched with a command like `python3 OrderedList.py`. The operations on the two lists have the same names but have different behaviors, of course, to maintain the ordering. [Figure 5-9](#) shows the tool with a list of ingredients already entered, ordered alphabetically.



Figure 5-9 *The `OrderedList` Visualization tool*

Type some words and insert them using the text entry box and the Insert button. The first item always goes in the first position, but subsequent items get placed after any items already in the list by their alphabetical order. Watch the insertion animation after it finds the right place in the list. It's nearly the same as for the unordered list.

With several items in your list, try a search for an item. Search for both an existing key and one that's not in the list. The search stops as soon as it finds an item with a key larger than that of the goal key. Next, try deleting a key that's early in the list. The deletion process is the same as the unordered one when the key is in the list, but the search process for missing keys ends after it finds higher valued keys.

Python Code for Ordered Lists

An ordered list is a special kind of linked list, so the `OrderedList.py` module shown in [Listing 5-13](#) defines the `OrderedList` class as a subclass of `LinkedList`. It has one extra attribute, `__key`, which is a function that extracts the sorting key from items in the list. The sort key is stored when the list is

created and cannot be changed because that would allow a list of existing items to become unordered with respect to the key.

Listing 5-13 *The Beginning of the `OrderedList.py` Module*

```
from LinkedList import *

class OrderedList(LinkedList): # An ordered linked list where items
    def __init__(self,          # are in increasing order by key, which
        key=identity): # is retrieved by a func. on each item
        self.__first = None    # Reference to first Link, if any
        self.__key = key       # Function to retrieve key

    def getFirst(self): return self.__first # Return the first link

    def setFirst(self, link): # Change the first link to a new Link
        if link is None or isinstance(link, Link): #Must be Link or None
            self.__first = link
        else:
            raise Exception("First link must be Link or None")

    def find(self, goal): # Find the 1st Link whose key matches
                        # or is after the goal
        link = self.getFirst() # Start at first link, and search
        while (link is not None and # while not end of the list
            self.__key(link.getData()) < goal): # and before goal
            link = link.getNext() # Advance to next in list
        return link # Return Link at or just after goal or None for end

    def search(self, goal): # Find 1st datum whose key matches goal
        link = self.find(goal) # Look for Link object that matches
        if (link is not None and # If Link found, and its key
            self.__key(link.getData()) == goal): # matches the goal
            return link.getData() # return its datum
```

Because `OrderedList` is a subclass of `LinkedList`, it inherits all the methods defined in [Listing 5-3](#) and [Listing 5-4](#). The `getFirst()` and `setFirst()` methods must be redefined here for the same reason we noted in the `DoubleEndedList` subclass: Python's name mangling of attributes that start with double underscores prevents sharing the parent class's attributes. The other methods like `getNext()` and `isEmpty()` work without rewriting them. Note that making `OrderedList` a subclass of `LinkedList` leaves `setFirst()` as

a public method, which is not advisable because callers could use it to place items in the list out of order.

The `find()` method resembles what was used for the `LinkedList`, but there are important changes. Because the list items are ordered, the methods that step through the list looking for a particular key take advantage of the ascending ordering. The method has a test in the `while` loop condition that checks that the key of the current link is less than that of the goal. That means that the loop can end when it

- Finds the link with the `goal` key
- Finds the next link immediately after where link with the goal key would go or
- Hits the end of the list

Because the loop can end with `link` pointing at either the desired `Link` or the one with the next higher key value, there's a question on what value to return when the `goal` is not found. This implementation returns either kind of `Link` because the capability of finding the one immediately following a particular key can be used for some operations (for example, splitting the list). This is like the behavior of the `find()` method for ordered record arrays in [Chapter 4](#), which returned the index where a new item should be inserted. In other words, the `find()` method returns a pointer to where a `Link` having the `goal` key *should be inserted in the list*. If it goes before the first item in the list, the pointer to the first `Link` is returned. If it goes somewhere before or at one of the other items, the pointer to that item is returned. Only if it goes after the last item is `None` returned.

The `search()` method calls `find()` and adds a test to its `if` statement checking whether the `Link` returned from `find()` is valid, and it returns the item stored in that `Link` only if its key matches.

The `insert()` method of the `OrderedList.py` module shown in [Listing 5-14](#) does something similar. Ideally, it would just use `find()` to locate the insertion point and modify the list there. In this case, however, it needs to modify the `__next` pointer of the `Link` preceding the one returned by `find()`. So, like the `delete()` method of `LinkedList`, it uses a loop to search for the previous `Link` to the insertion point. More precisely, it looks for either the `OrderedList` or `Link` that precedes the insertion point. The `while` loop condition changes to